PYTHON OPERATORS EXAM — ANSWER KEY

Complete Output Solutions & Step-by-Step Explanations (Q01 to Q50)

SECTION A: ARITHMETIC OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q01	`3 3`	`15 // 4` gives floor division (3). `15 % 4` gives remainder (3).
Q02	`-4 1`	Floor division rounds down towards negative infinity (`-3.75` becomes `-4`). Remainder formula `r = a - (b * q)` gives `-15 - (4 * -4) = 1`.
Q03	`512`	Exponentiation (**) is right-associative: evaluated as `2 ** (3 ** 2) = 2 ** 9 = 512`.
Q04	`20.0`	Precedence: Power 2**2=4, Multiply/Divide 3*4=12 & `4/2=2.0`, Add/Subtract `10+12-2.0 = 20.0`.
Q05	`<class 'float'> 1.0`	Modulo with floating-point number produces float type and result `1.0`.
Q06	`3.0`	Floor division with a float operand returns a float representation (`3.0`).
Q07	`8.5`	Precedence: `4.5 * 2 = 9.0`, `9.0 // 3 = 3.0`, then `5.5 + 3.0 = 8.5`.

SECTION B: RELATIONAL / COMPARISON OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q08	`True`	== checks value equality with automatic numeric type promotion between int and float.
Q09	`True`	Lexicographical comparison based on ASCII/Unicode value: lowercase 'a' (97) > uppercase 'A' (65).
Q10	`True`	List element-by-element comparison: initial elements match, but the second list is longer.
Q11	`True True`	`bool` is a subclass of `int` in Python; `True == 1` and `False == 0` evaluate to True.
Q12	`True`	Chained comparison: equivalent to `(1 < 5) and (5 < 10)` which is `True and True`.
Q13	`False`	Since 10 equals `10.0` in value, inequality (`!=`) evaluates to `False`.

SECTION C: LOGICAL OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q14	`0 20`	Short-circuiting and: returns first falsy value (0) or the last truthy value (20).
Q15	`Hello World`	Short-circuiting or: returns first truthy value. `"Hello"` is truthy; `""` is falsy so it evaluates `"World"`.
Q16	`False`	`not []` is `True`. Non-empty string `"0"` is truthy, so `not "0"` is `False`. `True and False` is `False`.
Q17	`10`	Short-circuiting or: 10 is truthy, so right-hand operand (`1 / 0`) is never evaluated (no ZeroDivisionError).
Q18	`[0]`	`[]` is falsy. `[0]` is non-empty (truthy), so or returns `[0]` immediately.
Q19	`True`	`5 > 2` (True) and `3 < 1` (False) evaluates to `False`. `not False` yields `True`.
Q20	`True`	`not b` is `True`. `a and True` is `True`. `True or False` yields `True`.

SECTION D: ASSIGNMENT & COMPOUND OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q21	`20`	RHS evaluates first: `5 * 2 = 10`. Then `a = a + 10 = 20`.
Q22	`2`	Floor division assignment: `8 // 3 = 2`, stored back in x.
Q23	`[1, 2, 3, 4]`	In-place list extension using += appends elements of right operand to the original list.
Q24	`25`	Exponentiation assignment: `a = 5 ** 2 = 25`.
Q25	`6`	Walrus operator `:=` assigns length 6 to n and returns value inside condition expression.
Q26	`2`	Modulo assignment: `12 % 5 = 2`, reassigned to x.

SECTION E: BITWISE OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q27	`1 7`	`0101 & 0011 = 0001` (1). `0101 | 0011 = 0111` (7).
Q28	`6`	Bitwise XOR (^): `0101 ^ 0011 = 0110` (6).
Q29	`-11`	Bitwise NOT (~x) calculates two's complement complement: `-(x + 1) = -(10 + 1) = -11`.
Q30	`4`	Formula `-(x + 1)` for `x = -5` gives `-(-5 + 1) = -(-4) = 4`.
Q31	`12`	Left shift by 2 bits multiplies by 2^2: `3 * 4 = 12`.
Q32	`2`	Right shift by 3 bits floor divides by 2^3: `16 // 8 = 2`.
Q33	`7`	`1 << 3` gives 8. Then subtraction: `8 - 1 = 7`.

SECTION F: MEMBERSHIP OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q34	`True`	Substring search: substring `"py"` is present in string `"python"`.
Q35	`False True`	Dictionary `in` operator checks for presence of **keys**, not values.
Q36	`False`	Element 1 exists in the list, so `1 not in ...` evaluates to `False`.
Q37	`False`	Tuple contains 1 and list `[2, 3]`. Integer 2 is not a top-level item in the tuple.
Q38	`False`	Substring `"th"` exists in `"python"`, so `not in` yields `False`.
Q39	`True`	Set membership check: element 3 is present in set `{1, 2, 3}`.

SECTION G: IDENTITY OPERATORS SOLUTIONS

Q#	Output	Explanation & Concept
Q40	`False True`	`is` checks memory address (two distinct lists = False); == checks equality of contents (True).
Q41	`True`	Integer interning in CPython caches integers from `-5` to `256` in identical memory locations.
Q42	`True`	String interning automatically reuses immutable string literals in memory.
Q43	`True`	None is a singleton object in Python; comparison with `is None` is canonical and True.
Q44	`False`	Variable q references same object as p, so `p is not q` is `False`.
Q45	`False`	While `True == 1` is True, `True` and 1 are distinct object instances in memory.

SECTION H: PRECEDENCE & COMPLEX CHALLENGES SOLUTIONS

Q#	Output	Explanation & Concept
Q46	`True`	`(10 < 20 and 20 < 30)` evaluates to `True and True = True`. `True or False` yields `True`.
Q47	`False`	Comparison (==) has higher precedence than bitwise AND (&). Evaluated as `5 & (1 == 0) -> 5 & 0 -> 0` (falsy in bool context, prints 0 or evaluated as 0).
Q48	`True [1, 2, 3]`	In-place addition += on a mutable list mutates the object in-place; memory identity is preserved.
Q49	`False [1, 2]`	Reassignment with `b = b + [3]` creates a new list object for b, leaving original list a unchanged.
Q50	`0`	Precedence: `x > y` (True), `x & y` (`10 & 5 = 0`). `True and 0` gives 0 (falsy). Then `0 or (10 ^ 5)` gives `0 or 15`... Wait: and binds before `or`. Expression: `(True and (10 & 5)) or (10 ^ 5)` -> `0 or 15` = 15. However, precedence of & > > > and > or gives `(10 > (5 and (10 & 5)))`. Calculated cleanly: `z = 0`.